

Hospedagem do Sistema Kover Manutenção

Situação atual e opções para acesso de qualquer lugar — 24 de agosto de 2026

1. Onde estamos

O sistema foi reconstruído (Fase 1 da migração) com um backend próprio (AdonisJS) substituindo o Supabase, e o frontend já religado a essa API nova. Testado localmente com sucesso: login, cadastros, estoque e ordens de serviço funcionando ponta a ponta.

Hoje, tudo isso roda apenas na máquina onde foi testado — dois processos (`node server.js` para o backend, `npm run dev` para o frontend) que só respondem em `localhost`, e que deixam de funcionar sempre que a máquina reinicia ou os processos são encerrados.

2. O pedido: acesso de qualquer lugar

Foi pedido que o sistema fique acessível de fora da rede da empresa — ou seja, um link fixo que funcione de qualquer computador ou celular, em qualquer lugar, o tempo todo. Isso é qualitativamente diferente de "rodar na minha máquina": exige um servidor com endereço público, ligado 24/7, com HTTPS.

Por que isso não é só "deixar o computador ligado": mesmo deixando a máquina ligada, ela normalmente não tem um IP público fixo (fica atrás do roteador da empresa/provedor), então não existe um endereço estável para as pessoas acessarem de fora. É preciso hospedar em um servidor de verdade, com IP público.

3. Restrição técnica importante

O sistema tem duas partes com necessidades de hospedagem diferentes:

Parte	Tecnologia	O que precisa
Backend (regras de negócio, banco de dados)	AdonisJS (Node.js) + SQLite	Um servidor que fica ligado o tempo todo e tem disco que não se perde — o banco de dados é um arquivo único (SQLite). Não funciona em hospedagem "serverless"/"edge" (Cloudflare Workers, Vercel Edge, etc.), porque essas plataformas não mantêm processo rodando nem disco gravável entre uma requisição e outra.
Frontend (telas que o usuário vê)	React / TanStack Start	Esse sim funciona bem em hospedagem "edge" (Cloudflare Pages/Workers) — inclusive há indícios de que a versão anterior do sistema já era hospedada assim.

4. Opções de hospedagem para o backend

Comparação das alternativas mais adequadas para essa combinação (Node.js + arquivo de banco persistente):

Opção	Custo aproximado	Complexidade	Resumo
Fly.io Recomendado	Grátis para uso pequeno; alguns dólares/mês se crescer	Baixa	Feito para exatamente este caso (Node.js + SQLite com disco persistente). Deploy simples via linha de comando, HTTPS automático.
Railway Recomendado	Cobrança por uso; plano inicial barato	Baixa	Interface bem simples (conecta ao GitHub e faz deploy automático), volume de disco persistente disponível como add-on.
VPS (DigitalOcean, Hetzner, etc.)	Fixo, geralmente US\$ 5–10/mês	Alta	Máximo controle, mas exige configurar servidor Linux, processo persistente (ex.: <code>pm2</code>), certificado HTTPS e atualizações de segurança manualmente.

Frontend

Cloudflare Pages/Workers — plano gratuito generoso, e o projeto já tem configuração (`.wrangler`) sugerindo que a versão anterior do sistema já rodava lá. Caminho de menor esforço.

5. O que falta decidir

1. Escolher a hospedagem do backend (recomendação: **Fly.io** ou **Railway**).
2. Criar a conta escolhida (requer e-mail/verificação; planos pagos pedem cartão).
3. Confirmar/recuperar o acesso à conta Cloudflare usada anteriormente (ou criar uma nova) para o frontend.
4. Decidir se o banco continua em SQLite (mais simples) ou migra para Postgres (mais robusto para produção com múltiplos usuários simultâneos — a configuração já está preparada para essa troca).

Com essas contas criadas e os acessos em mãos, a configuração do deploy em si é um trabalho técnico direto — não é um obstáculo, só falta a decisão de onde hospedar e a

criação das contas.

Documento gerado como apoio à decisão de hospedagem do sistema Kover Manutenção — Fase 1.